

# ARIJIT SOW

Software Engineer

Experience: 3.3 Yrs.

## Professional Summary

Specialized Software Developer focused on engineering highly scalable backend systems using **Java, Spring Boot, and Microservices**. I bridge the gap between theoretical computer science and real-world application by utilizing core **data structures (arrays, linked lists) and algorithmic efficiency (search/sort)** to resolve complex performance bottlenecks. Backed by a robust technical toolkit that includes **Kafka, Redis, databases, and JUnit/Mockito**, I consistently deliver clean, resilient architectures while proactively driving workflow optimizations and adapting to rapidly evolving technologies.

## Technical Skills

|                                       |                                                                                           |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Languages</b>                      | Java, SQL                                                                                 |
| <b>J2EE Technologies</b>              | Servlet, JSP and JDBC                                                                     |
| <b>Frameworks- Back End</b>           | Hibernate (JPA), Spring (Core, MVC, Boot), Microservice, Spring Data JPA, Spring Security |
| <b>Architecture &amp; Design</b>      | Microservices, Low Level Design (LLD), Event-Driven Architecture, SOLID Principle, ACID   |
| <b>Design Patterns</b>                | Singleton, MVC, DTO, DAO, Factory, Builder, SAGA, API Gateway, Resilience4j, DDD, SRP     |
| <b>Version Control System</b>         | Git with Gitlab                                                                           |
| <b>DBMS Applications</b>              | MySQL, PostgreSQL, Oracle DB, MongoDB                                                     |
| <b>DevOps &amp; Cloud Technology</b>  | Jenkins, Docker, Aws (S3, DynamoDB)                                                       |
| <b>Caching</b>                        | Redis                                                                                     |
| <b>Logging API</b>                    | Slf4j, ELK Stack, Zipkin                                                                  |
| <b>Code Quality Tool</b>              | SonarQube                                                                                 |
| <b>Application Servers</b>            | Tomcat Apache.                                                                            |
| <b>Build Automation Tool</b>          | Maven, Gradle                                                                             |
| <b>Other Tools</b>                    | Eclipse, Visual Studio Code, IntelliJ IDEA, pgAdmin4                                      |
| <b>API Documentation</b>              | Swagger API, Postman                                                                      |
| <b>Message Broker</b>                 | Apache Kafka                                                                              |
| <b>Cloud</b>                          | AWS                                                                                       |
| <b>Software Development Process</b>   | Agile Methodology                                                                         |
| <b>Testing</b>                        | JUnit 5, Mockito, Integration Testing                                                     |
| <b>Data Structure &amp; Algorithm</b> | Arrays, Strings, LinkedList, Sorting, Searching                                           |

## Projects

### 1. AutoSync Data Fabric

**Domain:** Automotive | **Client:** Airdit | **Duration:** January 2025 to ongoing

### Technology Stack:

- **Backend Technologies** : Java 17, Spring Boot
- **Architecture** : Microservices

- **Design Patterns** : DAO and DTO, Factory, Builder, Singleton, SAGA
- **Project Management Tool** : Jira
- **Version Control** : GitHub
- **Testing & Libraries** : Unit Testing (JUnit & Mockito) and Integration Testing
- **DB Application** : S3Bucket, Postgres
- **Logging API** : Slf4j
- **Build** : Maven
- **Cache** : Redis
- **Event Streaming** : Kafka
- **API Documentation** : Postman, Confluence
- **Tools Used** : IntelliJ

### Project Details:

The AutoSync Data Fabric is a high-throughput, configuration-driven integration platform engineered to bridge the gap between legacy OEM systems and the Automotive Retail Cloud. By abstracting complex, heterogeneous data sources—including vehicle telemetry, parts catalogs, and multi-currency invoices—into a single canonical data model, the platform enables seamless interoperability at scale. Utilizing an event-driven core powered by Apache Kafka and a SAGA-based distributed transaction engine, the system ensures data consistency across a multi-tenant environment, allowing for the rapid, code-free onboarding of global enterprise partners with enterprise-grade fault tolerance.

### Roles and Responsibilities:

- **Microservices Architecture:** Architected and maintained a multi-tenant, event-driven backend using **Java 17** and **Spring Boot**, orchestrating high-volume integration flows via **Kafka** for loose coupling and scalability.
- **Integration & Orchestration:** Engineered a canonical data model and a configuration-driven orchestration engine to standardize diverse OEM payloads (REST), enabling rapid, code-free onboarding for new enterprise partners.
- **Resilience & Distributed Transactions:** Implemented the **SAGA** design pattern alongside advanced fault-tolerance mechanisms—including Dead Letter Queues (DLQs), circuit breakers, and exponential backoff retries—to ensure guaranteed message delivery and outage recovery.
- **API Security & Governance:** Developed and secured highly available RESTful APIs, enforcing strict versioning, schema validation, OAuth2 & JWT authentication, and rate limiting.
- **Data Storage & Caching:** Managed complex master data pipelines using **PostgreSQL** for reliable, structured, and scalable data storage, along with **Redis** caching to optimize the performance of multi-step workflows.
- **Observability & Quality Assurance:** Established end-to-end production observability using **Slf4j**, distributed tracing, and correlation IDs, while maintaining strict code reliability through comprehensive testing with **JUnit** and **Mockito**.

## 2. OptiRoute Logistics

**Domain:** Logistics & Supply Chain | **Client:** Azentio | **Duration:** March 2023 to December 2024

### Technology Stack:

- **Back End Technologies** : Java 17, Spring Boot
- **Design Patterns** : DAO and DTO, Factory, Builder, Singleton
- **Frameworks** : Spring Framework
- **RDBMS Application** : PostgreSQL

- **Logging API** : Logback, Log4j2
- **Build Automation Tool** : Maven
- **API Documentation** : Postman, Swagger
- **Testing Frameworks** : JUnit5, Mockito
- **Build & Dependency Tool** : Maven
- **Cloud Storage** : AWS S3
- **Caching Technologies** : Redis
- **Tools Used** : IntelliJ, PG Admin

### Project Details:

OptiRoute Logistics is a high-performance logistics management platform engineered to automate and scale global supply chain operations. The system serves as a mission-critical bridge between shippers and a diverse network of transporters, utilizing an event-driven architecture to manage real-time logistics events, automated pricing rule execution, and dynamic rate-card configurations. By leveraging a distributed microservices ecosystem, the platform ensures rapid, secure onboarding of logistics partners while maintaining centralized operational visibility and rigorous compliance standards for high-volume cargo movement.

### Roles and Responsibilities:

- **Core Backend Engineering:** Designed and developed scalable, high-performance RESTful APIs using Java 17 and Spring Boot, driving the core backend logic for the transporter onboarding and management modules.
- **Architectural Design:** Implemented robust enterprise design patterns (including DAO, DTO, Factory, Builder, and Singleton) to ensure a decoupled, maintainable codebase and secure data transfer across distributed services.
- **Event-Driven Processing:** Engineered asynchronous, event-driven workflows to handle high volume, real-time logistics tracking and pricing events, significantly improving system throughput and responsiveness.
- **Database & Caching Optimization:** Designed complex relational schemas in PostgreSQL for rate-card and pricing rule management, and integrated Redis caching to reduce database load and accelerate API response times for operational dashboards.
- **Cloud Integration:** Integrated AWS S3 for the secure, scalable storage and fast retrieval of transporter compliance documentation and assets.
- **API Design & Documentation:** Spearheaded API lifecycle management, utilizing Postman and Swagger (OpenAPI) for clear technical documentation and Postman for endpoint validation and frontend integration testing.
- **Quality Assurance:** Achieved high code reliability and test coverage by authoring comprehensive unit and integration test suites utilizing JUnit 5 and Mockito.
- **Observability & Build:** Configured centralized, asynchronous application logging using Logback/Log4j2 for efficient production debugging, and managed project dependencies and builds using Maven.